

Product Requirements Document (PRD)

Project Title

Cloud-Based File Distribution System (CBFDS)

1. Project Overview

The Cloud-Based File Distribution System is a secure web application that enables users to upload, store, manage, share, and download files through cloud storage.

Instead of storing files as a single object, the system automatically divides every uploaded file into smaller chunks, stores them securely, records metadata for each chunk, and reconstructs the original file during download.

The project demonstrates important Cloud Computing concepts including scalability, distributed storage, redundancy, fault tolerance, metadata management, and secure file transfer.

2. Problem Statement

Traditional file storage systems become difficult to scale as storage demand increases. Large files also consume more bandwidth and are harder to recover if corruption occurs.

This project solves these problems by implementing:

- File chunking
 - Metadata management
 - Secure cloud storage
 - Reliable reconstruction
 - Integrity verification
 - Scalable architecture
-

3. Project Goals

The system should allow users to:

- Register and log in securely.
- Upload files of different formats.

- Automatically split uploaded files into chunks.
 - Store chunks securely.
 - Save metadata for every chunk.
 - Download files by reconstructing all chunks.
 - Share files with other users.
 - Manage uploaded files.
 - Delete or rename files.
 - Verify file integrity before download.
-

4. Target Users

- Students
 - Educational institutions
 - Small organizations
 - Personal cloud storage users
 - Software developers
 - Researchers
-

5. Functional Requirements

Authentication Module

The system shall provide:

- User Registration
 - User Login
 - JWT Authentication
 - Password Encryption
 - Logout
 - Forgot Password
 - Reset Password
-

User Dashboard

The dashboard shall display:

- Total files
- Storage used
- Recent uploads
- Recent downloads
- Shared files

- Available storage
-

File Upload Module

The system shall:

- Accept multiple file formats
 - Display upload progress
 - Validate file size
 - Validate file type
 - Generate a unique File ID
 - Store original file metadata
-

File Chunking Module

After upload the system shall:

- Split the file into configurable chunks
 - Assign a chunk number
 - Generate checksum for each chunk
 - Store chunk metadata
 - Upload chunks to storage
-

Storage Module

The storage system shall:

- Save chunks
 - Retrieve chunks
 - Delete chunks
 - Update chunk information
 - Replicate chunks (future enhancement)
-

Metadata Module

Metadata shall include:

- File ID
- Owner ID
- File Name
- File Size

- Upload Date
 - Number of Chunks
 - Hash Value
 - Storage Location
-

Download Module

The system shall:

- Retrieve all chunks
 - Verify checksums
 - Merge chunks
 - Reconstruct the original file
 - Send the completed file to the user
-

File Management

Users can:

- Rename files
 - Delete files
 - Search files
 - Sort files
 - Filter files
 - View upload history
-

Sharing Module

Users can:

- Generate share links
 - Set expiry dates
 - Disable links
 - Control permissions
 - Share with selected users
-

Admin Module

The administrator can:

- Manage users

- View system statistics
 - View storage usage
 - Delete inappropriate files
 - Monitor uploads
 - Monitor downloads
 - View activity logs
-

6. Non-Functional Requirements

The system should provide:

Performance

- Fast uploads
- Fast downloads
- Efficient chunk retrieval
- Support concurrent users

Security

- Password hashing
- JWT authentication
- HTTPS
- File validation
- Secure API endpoints
- Role-based access

Reliability

- Automatic recovery
- Metadata backup
- Error logging
- Integrity verification

Scalability

The system should support:

- Thousands of users
 - Millions of files
 - Horizontal storage expansion
-

7. Technologies

Frontend

- React
- Tailwind CSS
- Axios
- React Router

Backend

- Node.js
- Express.js

Database

- MongoDB

Authentication

- JWT
- bcrypt

File Upload

- Multer

Cloud Storage

Initially:

- Local Storage (Development)

Future:

- AWS S3
- Google Cloud Storage
- Azure Blob Storage
- MinIO

8. User Workflow

1. User registers.

2. User logs in.
 3. User uploads a file.
 4. Backend validates the file.
 5. File is divided into chunks.
 6. Chunks are stored.
 7. Metadata is saved.
 8. User sees upload success.
 9. User downloads a file.
 10. Backend retrieves all chunks.
 11. Chunks are merged.
 12. Original file is returned.
-

9. Future Enhancements

- Multi-cloud storage
 - AI storage optimization
 - Automatic backup
 - Version control
 - File synchronization
 - End-to-end encryption
 - Virus scanning
 - Storage analytics
 - Mobile application
 - Distributed storage nodes
-

10. Success Criteria

The project will be considered successful if it:

- Supports secure authentication.
 - Uploads files successfully.
 - Splits files into chunks.
 - Stores chunk metadata correctly.
 - Reconstructs files without data loss.
 - Verifies integrity using checksums.
 - Provides a responsive dashboard.
 - Handles multiple users reliably.
 - Is scalable for future cloud providers.
-

11. Constraints

- Initial deployment will use local storage or MinIO for development.
 - Maximum upload size will be configurable.
 - Internet connectivity is required for cloud operations.
 - Browser compatibility should include modern Chromium, Firefox, and Edge browsers.
-

12. Deliverables

- Responsive Web Application
- REST API
- MongoDB Database
- Authentication System
- File Chunking Engine
- Storage Service
- Metadata Management
- Admin Dashboard
- Technical Documentation
- Deployment Guide
- User Manual